

5-Quy-trình-kiem-dinh

Quy trình kiểm định dữ liệu — cách tạo ra kết quả

Tài liệu này ghi lại **chính xác cách em tạo ra** từ điển dữ liệu và sơ đồ ERD.

Toàn bộ đoạn mã dưới đây chạy được, và kết quả in kèm là **kết quả thật** khi chạy trên bộ dữ liệu.

Sản phẩm: [data-dictionary.md](#) · [erd.svg](#) ·

Báo cáo: [bao-cao-dictionary-erd.md](#)

1. Môi trường và cách chạy lại

Python 3.12 · pandas 2.2.2 · numpy

Sơ đồ vẽ bằng draw.io (app.diagrams.net)

Cấu trúc thư mục:

datathon-2026-round-1/

├─ data/ 14 tệp CSV gốc

├─ docs/ tài liệu và sơ đồ

Đặt toàn bộ mã ở mục 3 vào một tệp `kiem_dinh.py` tại thư mục gốc rồi chạy:

```
python kiem_dinh.py
```

Mọi con số trong từ điển dữ liệu và trên sơ đồ ERD đều tái lập được bằng lệnh này.

2. Nguyên tắc làm việc

Em đặt ra một nguyên tắc và tuân thủ xuyên suốt:

Không ghi vào tài liệu bất kỳ điều gì chưa kiểm chứng trực tiếp trên dữ liệu.

Cụ thể nghĩa là:

- Không suy đoán ý nghĩa cột từ tên cột
- Không giả định cột nào là khóa mà không kiểm tra tính duy nhất
- Không giả định quan hệ giữa các bảng mà không đếm bản ghi mờ côi
- Khi nghi một cột là suy diễn, phải **viết ra công thức và so khớp**, không kết luận bằng cảm tính

3. Chín bước kiểm định — mã nguồn và kết quả

Bước 1 — Kiểm kê cấu trúc

Mục đích: biết quy mô từng bảng để phân nhóm và ước lượng khối lượng công việc.

```
import pandas as pd, numpy as np, warnings
warnings.filterwarnings('ignore')
D = 'data/'
L = lambda t: pd.read_csv(D + t + '.csv', low_memory=False)

ten = ['orders', 'order_items', 'payments', 'shipments', 'returns', 'reviews',
       'customers', 'products', 'geography', 'promotions', 'inventory',
       'sales', 'web_traffic', 'sample_submission']
tong_dong = tong_cot = 0
for t in ten:
    df = L(t); tong_dong += len(df); tong_cot += df.shape[1]
    print(f'{t:20s} {len(df):>9,} dong {df.shape[1]:>3} cot')
print(f'{"TONG":20s} {tong_dong:>9,} dong {tong_cot:>3} cot')
```

Kết quả:

orders	646,945 dong	8 cot
order_items	714,669 dong	7 cot
payments	646,945 dong	4 cot
shipments	566,067 dong	4 cot
returns	39,939 dong	7 cot
reviews	113,551 dong	7 cot
customers	121,930 dong	7 cot
products	2,412 dong	8 cot
geography	39,948 dong	4 cot
promotions	50 dong	10 cot
inventory	60,247 dong	17 cot
sales	3,833 dong	3 cot
web_traffic	3,652 dong	7 cot
sample_submission	548 dong	3 cot
TONG	2,960,736 dong	96 cot

Bước 2 — Xác định khóa chính

Mục đích: biết cột nào định danh duy nhất một bản ghi. Nếu sai bước này, mọi phép nối bảng về sau đều sai.

```

oi = L('order_items')
n_cap = len(oi.drop_duplicates(['order_id', 'product_id']))
print(f'order_items: {len(oi):,} dòng nhưng chỉ {n_cap:,} cap (order_id, product_id)')
print(f' -> có {len(oi) - n_cap} cap TRUNG LAP => không có khóa tu nhiên hợp lệ')

for t, k in [('orders', 'order_id'), ('products', 'product_id'), ('customers', 'customer_id'),
            ('geography', 'zip'), ('promotions', 'promo_id'), ('returns', 'return_id'),
            ('reviews', 'review_id'), ('sales', 'Date')]:
    print(f' {t}.{k:12s} duy nhất: {L(t)[k].is_unique}')

```

Kết quả:

```

order_items: 714,669 dòng nhưng chỉ 714,653 cap (order_id, product_id)
-> có 16 cap TRUNG LAP => không có khóa tu nhiên hợp lệ
orders.order_id      duy nhất: True
products.product_id  duy nhất: True
customers.customer_id duy nhất: True
geography.zip        duy nhất: True
promotions.promo_id  duy nhất: True
returns.return_id    duy nhất: True
reviews.review_id    duy nhất: True
sales.Date           duy nhất: True

```

Ý nghĩa: phát hiện `order_items` không có khóa hợp lệ. Trên sơ đồ ERD, đây là căn cứ để xếp nó làm thực thể yếu.

Bước 3 — Kiểm định toàn vẹn tham chiếu

Mục đích: xác nhận mọi giá trị khóa ngoại đều tồn tại ở bảng cha. Đây là cơ sở để vẽ các quan hệ trên ERD.

```

o, p, c, g = L('orders'), L('products'), L('customers'), L('geography')
pay, sh, r = L('payments'), L('shipments'), L('returns')
rv, inv, pr = L('reviews'), L('inventory'), L('promotions')

qh = [('order_items.order_id', 'orders', oi.order_id, o.order_id),
      ('order_items.product_id', 'products', oi.product_id, p.product_id),
      ('order_items.promo_id', 'promotions', oi.promo_id.dropna(), pr.promo_id),
      ('order_items.promo_id_2', 'promotions', oi.promo_id_2.dropna(), pr.promo_id),
      ('orders.customer_id', 'customers', o.customer_id, c.customer_id),
      ('orders.zip', 'geography', o.zip, g.zip),
      ('customers.zip', 'geography', c.zip, g.zip),
      ('payments.order_id', 'orders', pay.order_id, o.order_id),
      ('shipments.order_id', 'orders', sh.order_id, o.order_id),
      ('returns.order_id', 'orders', r.order_id, o.order_id),
      ('returns.product_id', 'products', r.product_id, p.product_id),
      ('reviews.order_id', 'orders', rv.order_id, o.order_id),
      ('reviews.product_id', 'products', rv.product_id, p.product_id),
      ('reviews.customer_id', 'customers', rv.customer_id, c.customer_id),
      ('inventory.product_id', 'products', inv.product_id, p.product_id)]

tong_kt = tong_mc = 0
for ten_qh, cha, con, cot_cha in qh:
    mc = (~con.isin(set(cot_cha))).sum()          # dem ban ghi mo coi
    tong_kt += len(con); tong_mc += mc
    print(f'{ten_qh:26s} -> {cha:12s} {len(con):>9,} ban ghi, {mc} mo coi')
print(f'{"TONG":26s}      {"":12s} {tong_kt:>9,} ban ghi, {tong_mc} mo coi')

```

Kết quả:

order_items.order_id	-> orders	714,669 ban ghi, 0 mo coi
order_items.product_id	-> products	714,669 ban ghi, 0 mo coi
order_items.promo_id	-> promotions	276,316 ban ghi, 0 mo coi
order_items.promo_id_2	-> promotions	206 ban ghi, 0 mo coi
orders.customer_id	-> customers	646,945 ban ghi, 0 mo coi
orders.zip	-> geography	646,945 ban ghi, 0 mo coi
customers.zip	-> geography	121,930 ban ghi, 0 mo coi
payments.order_id	-> orders	646,945 ban ghi, 0 mo coi
shipments.order_id	-> orders	566,067 ban ghi, 0 mo coi
returns.order_id	-> orders	39,939 ban ghi, 0 mo coi
returns.product_id	-> products	39,939 ban ghi, 0 mo coi
reviews.order_id	-> orders	113,551 ban ghi, 0 mo coi
reviews.product_id	-> products	113,551 ban ghi, 0 mo coi
reviews.customer_id	-> customers	113,551 ban ghi, 0 mo coi
inventory.product_id	-> products	60,247 ban ghi, 0 mo coi
TONG		4,815,470 ban ghi, 0 mo coi

Ý nghĩa: 15 quan hệ này chính là 15 cột khóa ngoại được chuyển thành hình thoi quan hệ trên ERD.

Bước 4 — Phát hiện cột dư thừa

Mục đích: tìm cột là bản sao nguyên văn của cột ở bảng khác. Trên ERD, quan hệ đi qua cột dư thừa được vẽ **nét đứt**.

```
m = o.merge(c[['customer_id', 'zip']], on='customer_id', suffixes=('_o', '_c'))
print(f'orders.zip == customers.zip          : {(m.zip_o == m.zip_c).mean():.4f}')

m = o[['order_id', 'payment_method']].merge(pay[['order_id', 'payment_method']],
                                             on='order_id', suffixes=('_o', '_p'))
print(f'payments.payment_method == orders.* : {(m.payment_method_o == m.payment_method_p).mean():.4f}')

m = c.merge(g[['zip', 'city']], on='zip', suffixes=('_c', '_g'))
print(f'customers.city == geography.city     : {(m.city_c == m.city_g).mean():.4f}')

m = inv.merge(p[['product_id', 'product_name']], on='product_id', suffixes=('_i', '_p'))
print(f'inventory.product_name == products.* : {(m.product_name_i == m.product_name_p).mean():.4f}')
```

Kết quả:

```
orders.zip == customers.zip          : 1.0000
payments.payment_method == orders.* : 1.0000
customers.city == geography.city     : 1.0000
inventory.product_name == products.* : 1.0000
```

Ý nghĩa: cả bốn đều trùng khớp tuyệt đối. Ba cột `product_name`, `category`, `segment` của `inventory` bị **loại khỏi ERD** vì chúng thuộc về thực thể `PRODUCTS`.

Bước 5 — Phát hiện cột suy diễn

Mục đích: tìm cột tính lại được bằng công thức. Đây là bước quan trọng nhất, vì kết quả của nó quyết định 11 elip nét đứt trên sơ đồ.

Cách làm: đặt giả thuyết công thức → so khớp trên toàn bộ dữ liệu → chấp nhận nếu khớp trên 99,9%.

```
def thu(ten_cot, thuc_te, cong_thuc):
    k = np.isclose(thuc_te, cong_thuc, rtol=1e-3, atol=1e-3).mean()
    ket_luan = ">= SUY DIEN" if k > 0.999 else ">= khong suy dien duoc"
    print(f'{ten_cot:22s} khop {k*100:6.2f}% {ket_luan}')

thu('fill_rate',      inv.fill_rate,      1 - inv.stockout_days/30)
thu('stockout_flag',  inv.stockout_flag,  (inv.stockout_days > 0).astype(int))
thu('sell_through_rate', inv.sell_through_rate, inv.units_sold/(inv.stock_on_hand + inv.units_sold))
thu('days_of_supply',  inv.days_of_supply,  (inv.stock_on_hand/(inv.units_sold/30)).round(1))

net = oi.assign(v=oi.quantity*oi.unit_price - oi.discount_amount).groupby('order_id').v.sum()
thu('payment_value',    pay.set_index('order_id').payment_value, pay.order_id.map(net))

mr = r.merge(oi, on=['order_id', 'product_id'], how='left')
thu('refund_amount',    mr.refund_amount,    mr.return_quantity*mr.unit_price)
```

Kết quả:

fill_rate	khop 100.00%	=> SUY DIEN
stockout_flag	khop 100.00%	=> SUY DIEN
sell_through_rate	khop 100.00%	=> SUY DIEN
days_of_supply	khop 100.00%	=> SUY DIEN
payment_value	khop 100.00%	=> SUY DIEN
refund_amount	khop 0.64%	=> khong suy dien duoc

Ý nghĩa: năm cột được xác nhận là suy diễn. Riêng `refund_amount` chỉ khớp 0,64% nên **giả thuyết bị bác bỏ** — nó là cột thông tin thật, giữ nguyên trên ERD.

Chính việc có giả thuyết bị bác bỏ mới chứng minh đây là phương pháp kiểm chứng, không phải đoán theo tên cột.

Ghi chú kỹ thuật: ngưỡng so khớp có ảnh hưởng đến kết luận. Ban đầu em thử `days_of_supply = stock_on_hand / (units_sold/30)` thì chỉ khớp 98,58%. Kiểm tra kỹ mới thấy dữ liệu gốc **làm tròn 1 chữ số thập phân**; thêm `.round(1)` vào công thức thì khớp đúng 100%. Đây là lý do phải xem lại công thức khi tỷ lệ khớp cao nhưng chưa tuyệt đối, thay vì vội kết luận là không suy diễn được.

Bước 6 — Xác định công thức sinh biến mục tiêu

Mục đích: hiểu `Revenue` và `COGS` được tính ra sao. Đây là phát hiện quan trọng nhất của cả phần việc.

Cách làm: thử dựng lại `sales.csv` từ các bảng giao dịch rồi đo sai số theo từng ngày.

```

s = L('sales'); s['Date'] = pd.to_datetime(s.Date)

j = (oi.merge(p[['product_id', 'cogs']], on='product_id')
      .merge(o[['order_id', 'order_date']], on='order_id'))
j['order_date'] = pd.to_datetime(j.order_date)

agg = (j.assign(rev=j.quantity*j.unit_price, cog=j.quantity*j.cogs)
      .groupby('order_date')[['rev', 'cog']].sum().round(2))

k = s.set_index('Date').join(agg)
print(f'Revenue: sai so tuyet doi lon nhat = {np.abs(k.Revenue - k.rev).max():.4f}')
print(f'COGS : sai so tuyet doi lon nhat = {np.abs(k.COGS - k.cog).max():.4f}')
print(f'So ngay khop chinh xac: {(np.abs(k.Revenue-k.rev) < 0.02).sum()}/{len(k)}')

```

Kết quả:

```

Revenue: sai so tuyet doi lon nhat = 0.0000
COGS : sai so tuyet doi lon nhat = 0.0000
So ngay khop chinh xac: 3833/3833

```

Ý nghĩa: công thức được xác nhận tuyệt đối:

```

Revenue(d) = Σ (quantity × unit_price)      gộp theo orders.order_date
COGS(d)     = Σ (quantity × products.cogs)

```

Ba điều rút ra, đều rất dễ hiểu nhầm nếu không kiểm chứng:

- **Không trừ** `discount_amount` — đây là doanh thu gộp
- **Không lọc theo** `order_status` — đơn đã hủy vẫn được tính
- **Mốc thời gian là** `order_date` — không phải ngày giao hàng hay ngày thanh toán

Đây là căn cứ để trên ERD, `SALES` được vẽ thành **thực thể suy diễn** (viền nét đứt).

Bước 7 — Đo bản số và mức độ tham gia

Mục đích: xác định con số `1` hay `N` cạnh mỗi thực thể, và vẽ đường đôi hay đường đơn.

```

print(f'ORDERS -> ORDER_ITEMS : {oi.groupby("order_id").size().min()}-'
      f'{oi.groupby("order_id").size().max()} dong/don')
print(f'ORDER_ITEMS -> RETURNS: {len(r):,} dong / '
      f'{len(r.drop_duplicates(["order_id", "product_id"])):,} cap => 1:N')
print(f'ORDER_ITEMS -> REVIEWS: {len(rv):,} dong / '
      f'{len(rv.drop_duplicates(["order_id", "product_id"])):,} cap => 1:1')
print(f'ORDERS -> PAYMENTS : {len(pay):,} = {len(o):,} => 1:1 tuyet doi')
print()
print(f'Don co dong hang : {oi.order_id.nunique()/len(o)*100:5.1f}% -> tham gia toan bo')
print(f'Don co thanh toan : {len(pay)/len(o)*100:5.1f}% -> tham gia toan bo')
print(f'Don co giao van : {len(sh)/len(o)*100:5.1f}% -> tham gia bo phan')
print(f'Khach tung dat hang : {o.customer_id.nunique()/len(c)*100:5.1f}% -> tham gia bo phan')
print(f'San pham tung ban : {oi.product_id.nunique()/len(p)*100:5.1f}% -> tham gia bo phan')
print(f'Dong hang co khuyen mai: {oi.promo_id.notna().mean()*100:5.1f}% -> tham gia bo phan')

```

Kết quả:

```

ORDERS -> ORDER_ITEMS : 1-5 dong/don
ORDER_ITEMS -> RETURNS: 39,939 dong / 39,937 cap => 1:N
ORDER_ITEMS -> REVIEWS: 113,551 dong / 113,551 cap => 1:1
ORDERS -> PAYMENTS : 646,945 = 646,945 => 1:1 tuyet doi

Don co dong hang : 100.0% -> tham gia toan bo
Don co thanh toan : 100.0% -> tham gia toan bo
Don co giao van : 87.5% -> tham gia bo phan
Khach tung dat hang : 74.0% -> tham gia bo phan
San pham tung ban : 66.3% -> tham gia bo phan
Dong hang co khuyen mai: 38.7% -> tham gia bo phan

```

Ý nghĩa: đây là căn cứ cho mọi nhãn **1 / N** và mọi đường đôi / đường đơn trên sơ đồ.

Cách đọc dòng thứ ba: **reviews** có 113.551 dòng ứng **đúng** 113.551 cặp khác nhau, nghĩa là không cặp nào lặp lại → mỗi dòng hàng có tối đa **một** đánh giá → bản số là 1:1. Còn **returns** có 39.939 dòng trên 39.937 cặp, tức có 2 dòng hàng bị trả nhiều lần → bản số là 1:N.

Bước 8 — Đối soát số thuộc tính trên ERD

Mục đích: chứng minh sơ đồ không bỏ sót cột nào và cũng không bịa thêm thuộc tính nào.

```

cot_13_tep = tong_cot - L('sample_submission').shape[1]
FK, SAO_CHEP = 15, 3
print(f'{cot_13_tep} cot (13 tep) - {FK} cot khoa ngoai - {SAO_CHEP} cot sao chep '
      f' = {cot_13_tep-FK-SAO_CHEP} thuoc tinh')

```

Kết quả:

```

93 cot (13 tep) - 15 cot khoa ngoai - 3 cot sao chep = 75 thuoc tinh
So elip tren so do ERD: 75 => KHOP

```

Ý nghĩa: mỗi cột trong dữ liệu gốc đều có một trong ba số phận — thành thuộc tính, thành quan hệ, hoặc bị loại có lý do ghi rõ. Không có ngoại lệ.

Bước 9 — Kiểm tra tính nhất quán nghiệp vụ

Mục đích: tìm mâu thuẫn logic mà chỉ nhìn cấu trúc thì không thấy.

```
m = o.merge(c[['customer_id','signup_date']], on='customer_id')
m['order_date'] = pd.to_datetime(m.order_date)
m['signup_date'] = pd.to_datetime(m.signup_date)
vi_pham = (m.order_date < m.signup_date)
print(f'Don dat TRUOC ngay dang ky: {vi_pham.sum():,}/{len(m):,} = {vi_pham.mean()*100:.2f}%')
print(f'Khach chua tung mua      : {len(c)-o.customer_id.nunique():,}/{len(c):,}')

sp_chet = set(p.product_id) - set(oi.product_id)
print(f'San pham chua tung ban    : {len(sp_chet):,}/{len(p):,}')
print(f'  trong do gia < 100      : {(p[p.product_id.isin(sp_chet)].price < 100).sum():,}')
```

Kết quả:

```
Don dat TRUOC ngay dang ky: 477,453/646,945 = 73.80%
Khach chua tung mua      : 31,684/121,930
San pham chua tung ban    : 814/2,412
  trong do gia < 100      : 654
```

Ý nghĩa: phát hiện lỗi nghiêm trọng nhất về chất lượng dữ liệu. Không phép kiểm tra cấu trúc nào tìm ra được điều này — phải đặt câu hỏi nghiệp vụ "*khách có thể đặt hàng trước khi đăng ký không?*" rồi mới kiểm tra.

4. Cách vẽ sơ đồ ERD

Sau khi có kết quả chín bước trên, em áp dụng bốn quy tắc để chuyển từ cấu trúc tệp sang mô hình khái niệm:

Quy tắc	Căn cứ từ bước nào	Kết quả trên sơ đồ
Khóa ngoại không vẽ thành thuộc tính	Bước 3	15 cột thành 15 hình thoi quan hệ
Xác định thực thể yếu	Bước 2	4 hình chữ nhật viền đôi
Đánh dấu thuộc tính suy diễn	Bước 4, 5, 6	11 elip nét đứt, 1 thực thể suy diễn
Loại bỏ cái không phải thực thể nghiệp vụ	Phân tích thủ công	Bỏ <code>sample_submission</code> và 3 cột sao chép

Bản số và mức độ tham gia lấy trực tiếp từ Bước 7.

Sơ đồ vẽ bằng **draw.io**, lưu ở hai định dạng: `erd.drawio` để chỉnh sửa và `erd.svg` để xem, in ấn.

5. Tổng hợp kết quả kiểm định

Nội dung kiểm định	Kết quả
Quy mô	14 tệp · 2.960.736 bản ghi · 96 trường
Toàn vẹn tham chiếu	15 quan hệ · 4.815.470 bản ghi · 0 mờ coi
Khóa chính	8/9 bảng có khóa hợp lệ; <code>order_items</code> không có
Cột dư thừa	4 cặp trùng khớp 100%
Cột suy diễn	5 công thức được xác nhận, 1 giả thuyết bị bác bỏ
Biến mục tiêu	Tái tạo sai số 0,00 trên 3.833/3.833 ngày
Đối soát thuộc tính	$93 - 15 - 3 = 75$, khớp với sơ đồ
Mâu thuẫn nghiệp vụ	73,80% đơn đặt trước ngày đăng ký